

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №3

Выполнил:

Грищук Елизавета Дмитриевна
БР 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать контейнеризацию микросервисного приложения бронирования столиков в ресторанах средствами Docker. Написать Dockerfile для каждого сервиса, создать общий docker-compose.yml и настроить сетевое взаимодействие между сервисами.

Ход работы

Для каждого из 5 сервисов реализован Dockerfile на базе образа node:20-alpine. Образ собирается в несколько шагов: установка зависимостей, копирование исходного кода, сборка TypeScript в JavaScript через tsc, запуск скомпилированного приложения.

```
auth-service >  Dockerfile > ...  
1 FROM node:20-alpine  
2 WORKDIR /app  
3 COPY package*.json ./  
4 RUN npm install  
5 COPY tsconfig.json ./  
6 COPY src ./src  
7 RUN npm run build  
8 EXPOSE 3001  
9 CMD ["node", "dist/index.js"]
```

Написан общий docker-compose.yml, включающий 10 контейнеров: 4 базы данных PostgreSQL (auth_db, restaurant_db, booking_db, review_db), брокер сообщений RabbitMQ, и 5 сервисов (auth-service, restaurant-service, booking-service, review-service, api-gateway).

```
docker-compose.yml
  Run All Services
  Run Service
1  services:
2    auth-db:
3      image: postgres:15-alpine
4      environment:
5        POSTGRES_USER: postgres
6        POSTGRES_PASSWORD: liza
7        POSTGRES_DB: auth_db
8      networks:
9        - microservices-network
10     healthcheck:
11       test: ["CMD-SHELL", "pg_isready -U postgres"]
12       interval: 5s
13       timeout: 5s
14       retries: 5
15
```

Для обеспечения корректного порядка запуска реализованы healthcheck для каждой базы данных через pg_isready. Сервисы запускаются только после того как соответствующая база данных переходит в статус healthy.

Containers [Give feedback](#)

Container CPU usage ⓘ
8.10% / 800% (8 CPUs available)

Container memory usage ⓘ
274.46MB / 7.47GB

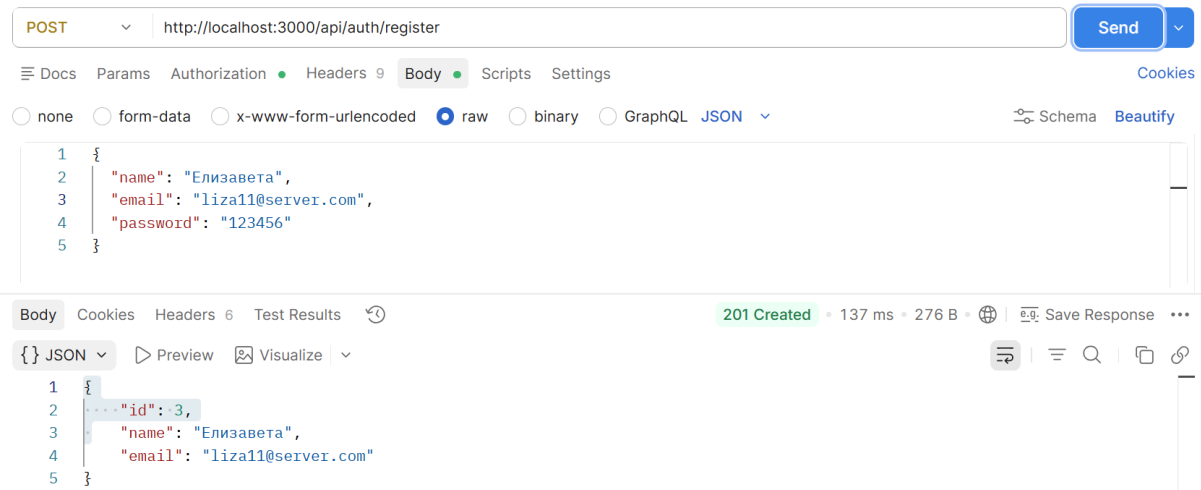
Show charts

☒ Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	▼	restaurant-micro	-	-	-	8.1%	12 hc	
☐	●	auth-service-	033395fbfbe9	restaurant-	3001:3001	0%	12 hc	
☐	●	restaurant-se	5bec74e035ec	restaurant-	3002:3002	0%	12 hc	
☐	●	restaurant-db	3cb05ac01cdd	postgres:1		3.74%	12 hc	

Все контейнеры объединены в общую сеть microservices-network с драйвером bridge. Межсервисное взаимодействие осуществляется по именам контейнеров (например http://auth-service:3001), что обеспечивает DNS-резолвинг внутри сети Docker.

Переменные окружения (порты, credentials БД, JWT_SECRET, AMQP_URL) передаются через секцию environment в docker-compose.yml.



Вывод

В ходе выполнения лабораторной работы реализована полная контейнеризация микросервисного приложения. Написаны Dockerfile для каждого из 5 сервисов, создан общий docker-compose.yml с 10 контейнерами. Настроено сетевое взаимодействие через bridge-сеть и healthcheck для баз данных. Все контейнеры успешно запускаются командой docker-compose up --build и API работает корректно через gateway на порту 3000.